

Set - II

① Back End of a compiler

Aim

To Write a C program to implement the back end of a compiler by generating simple target code from Three Address code.

Algorithm

- * Start
- * Read the number of Three Address code Statements.
- * Read each statement
- * Display the original Three Address code.
- * Convert each Statements into simple assembly like target code.
- * Display the generated target code.
- * Stop

C Program

```
#include <stdio.h>
```

```
int n, i;
```

```
char var[10], op1[10], op[5], op2[10];
```

```
printf("Enter number of statements:");
```

```
scanf("%d", &n);
```

```
printf("\n Enter Three Address code:\n");
```

```
for (i=0; i<n; i++) {
```

```
    scanf("%s = %s %s %s", var, op1, op,
```

```
    op2);
```

```
    printf("\n MOV R0, %s", op1);
```

```
    if (op[0] == '+')
```

```
        printf("\n ADD R0, %s", op2);
```

```
    else if (op[0] == '-')
```

```
        printf("\n SUB R0, %s", op2);
```

```
    else if (op[0] == '*')
```

```
        printf("\n MUL R0, %s", op2);
```

```
    else if (op[0] == '/')
```

```
        printf("\n DIV R0, %s", op2);
```

```
    printf("\n MOV %s, R0\n", var);
```

```
}
```

```
return 0;
```

```
}
```


input

2

$t_1 = a + b$

$t_2 = t_1 * c$

Output

MOV R0, a

ADD R0, b

MOV t1, R0

MOV R0, t1

MUL R0, c

MOV t2, R0

Result

Thus, the C program to implement the back end of a compiler was executed successfully.

Q2 Common Subexpression Elimination

Aim

Step. To write a C program to perform Common Subexpression Elimination on the given Three address code.

Algorithm:-

- * Start
- * Read the number of statements
- * Read each Three Address code statement.
- * Store each arithmetic expression.
- * Check whether the same expression was already computed.
- * If repeated, replace it with the previous temporary variable.

- * otherwise, store the expression normally.
- * display the original and optimized code.
- * stop.

C Program

```
#include <stdio.h>
#include <string.h>

int main() {
    int n, i, j;
    char lhs[10], op1[10], op[5], op2[10];
    char expr[20][30], temp[20][10];
    int count = 0;

    printf("Enter number of statements:");
    scanf("%d", &n);

    printf("\n Enter three Address code :\n");
    for (i = 0; i < n; i++) {
        scanf("%s = %s %s %s", lhs, op1, op, op2);
        printf("\n original : %s = %s %s %s\n",
            lhs, op1, op, op2);
        sprintf(expr[count], "%s %s %s", op1,
            op, op2);
        for (j = 0; j < count; j++) {
            if (strcmp(expr[count], expr[j]) == 0) {
```

```
printf("optimized : %s = %s\n", lhs,
```

```
temp[j]);
```

```
break;
```

```
}
```

```
}
```

```
if(j == count) {
```

```
strcpy(temp[count], lhs);
```

```
printf("optimized : %s = %s %s %s\n", lhs, op,
```

```
op, op2);
```

```
count++;
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

input

5

$t1 = a + b$

$t2 = c * b$

$t3 = a + b$

$t4 = t3 + t2$

$x = t4 + 0$

Result

output

original: $t1 = a + b$

optimized: $t1 = a + b$

original: $t2 = c * d$

optimized: $t2 = c * d$

original: $t3 = a + b$.

optimized: $t3 = t1$

original: $t4 = t3 + t2$

optimized: $t4 = t3 + t2$

original: $x = t4 + 0$

optimized: $x = t4 + 0$

Thus, the C program to perform Common Subexpression Elimination was executed successfully.

Q3

LEX program to Identify Integer Constants

Aim

To Write a Lex program to identify and print all integer constants from a C source program.

Algorithm

- * Start
- * Define a regular expression for integer constants
- * Read the input C source file.
- * Match all integer values.
- * Print each integer constant
- * Stop.

Lex program

```
%{  
#include <stdio.h>  
%}  
%%  
[0-9]+ { printf ("Integer constant: %s\n",  
                yy_text); }  
.  
%%  
int main() {  
    yyllex();  
    return 0;  
}
```

input : sample.c

```
int a = 10;
```

```
int b = 25;
```

```
int c = a + b;
```

output

Integer constant : 10

Integer constant : 25

Commands

```
lex integer.l
```

```
cc lex.yy.c -o integer
```

```
integer < sample.c
```

Result

Thus, the Lex program to identify and print integer constants from a C source program was executed successfully.

Q4. Lex program to identify Invalid Identifiers

Aim

To write a Lex program to identify invalid identifiers based on C identifiers rules.

Algorithm

- * Start
- * Read the input from the file.
- * Check each word.
- * A valid C identifier must start a letter or _.
- * The remaining characters can contain letters, digits, or _.
- * If the identifier starts with a digit or contains invalid symbols, print it as invalid.
- * Stop.

Lex program

```
%{
```

```
#include <stdio.h>
```

```
%}
```

```
%.
```

```
^[0-9][a-zA-Z0-9]* { printf("Invalid  
Identifier: %s\n", yytext); }
```

```
^[a-zA-Z][a-zA-Z0-9]* { printf("Valid Identifier:  
%s\n", yytext); }
```

```
[a-zA-Z][a-zA-Z0-9]*-[a-zA-Z0-9]+ { printf("Invalid  
Identifier: %s\n", yytext); }
```

```
}
```

```
ln ;
```

```
[14];
```

```
.
```

```
%.
```

```
int main () {
```

```
    yylex();
```

```
    return 0;
```

```
}
```

Input file: Sample.txt

total
_count
2value
Student1
mark-1

output

Valid Identifier : total

Valid Identifier : _count

Invalid Identifier : 2value

Valid Identifier : Student1

Invalid Identifier : mark-1

Command

lex identifier .1

cc . lex . yy . c - o identifier

identifier < sample . txt.

Result.

Thus, the Lex program to identify valid and invalid C identifiers was executed successfully.